

Cryptography!

- **This is a bonus lecture, which will not be on homework or tested**
- So why do we cover this?
- Because this is a discrete mathematics class in the computer science department
- This is one of the best examples of an interesting connection between mathematics and real-world computing

Private key cryptography (used since the Romans)

- Most fundamental task in cryptography: encrypt a message m so that:
 - m can be “obfuscated” or made unreadable to an adversary
 - But the intended recipient of the message can decrypt m .
- Most obvious encryption is called “**private key**” or “**symmetric**” cryptography
- In which Alice chooses a private key, k , that she shares only with intended recipient, Bob.
- Alice then encrypts her message with k and sends message to Bob
- Bob uses k to decrypt Alice’s message

Example of private key encryption (without proof) – sometimes called the “one-time pad”

- **Setting:** Alice has an n bit message $m \in \{0,1\}^n$ that must be sent to Bob!
- **Encryption:**
 - Alice chooses uniformly random key $k \in_R \{0,1\}^n$ and shares it with Bob via trusted courier.
 - Then Alice encrypts her message m by taking the “bitwise xor of m and k ”
 - i.e., bitwise xor of two bits $a \oplus b = 1$ iff exactly one of the bits is 1
 - So Alice sends to Bob $y = m \oplus k$ (that is, y is the n bit string so that $y_i = m_i \oplus k_i$ for all i)
- **Decryption:**
 - What can Bob, who holds the encrypted message, y , and k , do to get m ?
 - Bob takes the bitwise xor of y and k , and:
 - $m = y \oplus k = m \oplus k \oplus k$ since $k \oplus k = 0^n$
- Moreover, not hard to show that Eve (the evesdropper) who doesn't know k , cannot discover k from $y = x \oplus k$ in reasonable amount of time. Why?
 - Can show from Eve's perspective, $y = x \oplus k$ is totally uniform over all n bit strings! So she may as well guess the key randomly.

Public key cryptography (1970's)

- Problem with private key cryptography is that in practice...
 - Don't want a trusted courier to deliver the private key.
 - This is generally infeasible (how would Amazon.com work with this?)
- Alternative: public key, or “**asymmetric**” cryptography.
- In public-key cryptography there are two different keys
 - One is public (everyone can share!) that is used to encrypt
 - One is private that is used to decrypt
- How does this fix internet e-commerce applications?
 - E.g., if Amazon.com gives you their public key, you can use to encrypt credit card information to send to them.
 - Once you encrypt, no one but Amazon.com can decrypt!

RSA public-key encryption

- Most famous public-key encryption scheme is due to Rivest, Shamir, Adleman from the 70's.
- It uses ideas from number-theory: in particular, a generalization of Fermat's Little theorem called Euler's theorem.
- Euler's theorem:
 - For n any positive integer, let $\phi(n)$ be the number of numbers up to n that are rel. prime to n
 - *Not hard to see:* if $n = p \cdot q$ for primes p and q can show $\phi(n) = (p - 1)(q - 1)$
 - Recall **Fermat's Little Theorem**: If n rel. prime to a and a not divisible by n , $a^{n-1} \cong 1 \pmod{n}$
 - **Euler's theorem** (generalizes FLT): If n rel. prime to a , then $a^{\phi(n)} \cong 1 \pmod{n}$
- **Theorem [RSA'77]**: Assume given $n = p \cdot q$, it's hard to compute $\phi(n) = (p - 1) \cdot (q - 1)$. Then public-key encryption is possible!
 - This is actually a deep assumption.
 - In particular, it's assuming that factoring is a hard problem!

RSA encryption scheme

- **Theorem** [RSA'77]: Assume given $n = p \cdot q$, it's hard to compute $\phi(n) = (p - 1) \cdot (q - 1)$. Then public-key encryption is possible!
- **Key generation:** Choose two distinct, random primes p, q (very large!!)
 - Compute $n = p \cdot q$.
 - This will be the modulus for both public and private keys.
 - Choose integer e so that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$
 - This will be public!
 - Because e is rel. prime to $\phi(n)$...
 - There exists a mult. inverse, d so that $d \cdot e = 1 \bmod \phi(n)$.
 - **Public key:** (n, e)
 - **Private key:** (n, d) .
- **Encryption:** Alice share public key (n, e) with everyone (including Bob)
 - Say Alice wants to send message m to Bob.
 - Let's say $m \ll n$
 - Alice computes $c = m^e \bmod n$ and sends to Bob.
- **Decryption:** Bob receives $c = m^e \bmod n$ and holds (n, e, d) .
 - Bob computes: $c^d \bmod n = (m^e)^d \bmod n = m^{ed} \bmod n$ (*claim this equals m*)
 - Recall $ed = 1 \bmod \phi(n) \Rightarrow \phi(n) | (d \cdot e - 1) \Rightarrow k \cdot \phi(n) + 1 = d \cdot e$ by construction
 - So plugging in from above:
 - $m^{ed} \bmod n = m^{k\phi(n)+1} \bmod n = m(m^{k\phi(n)}) \bmod n = m(1) \bmod n = m \bmod n$
 - Where last equality is from Euler's theorem – generalized Fermat's little theorem!!
 - But this requires m to be rel. prime with n – true with high probability!

But is RSA really secure?

- RSA encryption, or similar ideas, are now completely ubiquitous on the internet!!
- But they are based on the assumption that can't factor efficiently.
- Is this really true?
- It may be true for today's computers, but it turns out that...
 - The physics of computation really matters here! Computers of the future that could take advantage of quantum mechanics – called “quantum computers” can factor numbers efficiently.
 - We are seeing the first examples of quantum computers **now**, although factoring will not be a practical task for these computers for many years to come
- **Open question in CS:** what will replace RSA, that will be secure against futuristic quantum computers?